Pre-Interview Take-Home Exercise

Senior AI Engineer - Regulatory Clause & Entity Extraction

We would like you to prepare a solution design for the scenario below, together with a small practical proof-of-concept for one focused part of your approach. The aim is to understand how you reason about an ambiguous AI engineering problem, make technical trade-offs and turn an idea into something reliable enough to operate in production.

Exercise overview

Area	Details
Role	Senior AI Engineer
Your task	Design a reliable AI system that ingests regulatory documents and extracts structured clauses and legally significant entities, each linked to source evidence and a confidence or review signal. Include a small code-based proof-of-concept for one focused part of your design.
Sample document	1a82a302-3d0a-49ee-992d-f0031aaeaf48.pdf
Expected output	A concise design document or short deck that makes the end-to-end system, quality bar and path to production plus a small script, notebook or similarly compact implementation artifact for one part of the proposed system.
Interview discussion (if solution is accepted)	90 minutes. Be ready to walk through your submission, the decisions and alternatives you considered, failure modes, how you would take it into production, and relevant experience from systems you have shipped or operated.
What we look for	Problem decomposition, AI engineering judgment, trust and evaluation, production thinking, practical implementation and clear communication.

1. Context and scenario

Adherent (formerly Compliance & Risks) helps global enterprises monitor, assess and prove compliance across many markets and jurisdictions. As a Senior AI Engineer you will build production-grade AI that converts complex regulatory text into structured, trustworthy intelligence that downstream risk, compliance, and workflow systems can act on. This exercise is a small, representative example of that problem.

You are given a regulatory document (for example a regulation, directive, standard or enforcement notice as PDF). It may contain definitions, scope, obligations, exemptions, deadlines, penalties, tables, footnotes, and cross-references. We want a system that, for documents like this at scale and across jurisdictions, can reliably:

- Segment and extract important clauses or sections such as definitions, responsibilities, requirements, exemptions, penalties and effective dates.
- Extract legally significant entities such as organisations, individuals, locations, dates, monetary thresholds, product categories, and references to other regulations or standards and link them to the relevant clauses.

- Attach source evidence to extracted information and expose an appropriate confidence or review signal.
- Produce outputs that can be used reliably by downstream compliance, risk, or workflow systems.

2. Assumptions

You may use the assumptions below or replace them with your own. State any assumptions that materially affect your solution.

Area	Suggested assumption
Volume	Thousands of regulatory documents per jurisdiction; new and changed documents arrive continuously, with occasional bulk backfills.
Document profile	5 to 300+ pages; mixed PDF and HTML; inconsistent structure, tables, footnotes, cross-references, and some scanned pages.
Languages	Primarily English to begin with, without preventing later multilingual expansion.
Accuracy	High precision matters more than latency. High-impact or low-confidence extractions may require human review before publication.

3. What to submit

Your submission should make your reasoning and choices clear without trying to cover every possible technique. Use your judgment on the level of depth and finish that is appropriate. There is no prescribed time limit for the exercise.

1. A concise solution design explaining how you would approach the problem end-to-end. We should be able to understand the shape of the system and the path from source document to usable output at a glance, as well as the major technical choices behind it. Explain what evidence and evaluations would make you comfortable that the system is working as intended and how you would release, operate and evolve it in production. Make clear what you would build first, what you would defer and the main risks or trade-offs you see.
2. A small practical proof-of-concept for one part of your proposed solution that you consider important or high-risk. Use the supplied document or a subset of it and make that part of the design concrete in code. A script, notebook or small repository (even pseudo-code) is sufficient. It should produce representative output that we can inspect.
3. Any supporting material needed to understand the submission, such as sample output, assumptions, or brief README notes. Include only what you believe adds value.

Keep the practical component narrow. We are not expecting a full implementation, user interface or production deployment.

4. What we are looking for

- How you decompose a problem and decide what matters first.
- How you think about reliability, evidence, evaluation, uncertainty, and appropriate human review.
- How you would make the system deployable, observable, maintainable and scalable in production.
- The practicality and clarity of the implementation artifact you choose to build.
- Clear communication, ownership of decisions and the ability to discuss alternatives and limitations.

5. Guidance and interview discussion

- Use any language, model, provider, framework or AI coding assistant you are comfortable with and cite the usage wherever relevant. Do not share API keys, credentials or other secrets.
- If anything is unclear, state a reasonable assumption and proceed.
- The interview is a collaborative deep-dive into your submission. We may change a constraint or ask you to explore an alternative verbally, but there is no separate live coding or algorithm-style assessment.
- We care about your reasoning and judgment more than presentation polish or the amount of code produced.